Grado en Inteligencia Artificial · Universidade da Coruña · Curso 2026/27

Práctica 2

Seguimiento de un vehículo en la ciudad de Robobo con neuroevolución

Evolución de controladores neuronales con NEAT. Desarrollo en simulación, comparación con el aprendizaje por refuerzo y transferencia al entorno real

Entrega	**Domingo 8 de noviembre de 2026**, por el campus online
Duración	Tres semanas, del 19 de octubre al 8 de noviembre
Formato	Archivo .zip con (1) memoria, (2) declaración de uso de IA, (3) código fuente comentado y (4) evidencias: genomas, configuración de NEAT, registros y vídeos
Memoria	Máximo 6 páginas, más los anexos que hagan falta, en Calibri 11 pt con interlineado sencillo. La declaración de uso de IA es un documento aparte, de una página como máximo
Trabajo	Por parejas

1. Objetivo

Esta práctica resuelve la misma tarea que la Práctica 1: que un Robobo, el **seguidor**, circule detrás del **vehículo guía** en el escenario "Follow City" de RoboboSim, **manteniendo la separación dentro de la banda objetivo, sin chocar, sin perderlo de vista y sin salirse de su carril** (este último requisito con una tolerancia). Lo que cambia es la forma de obtener el controlador: en lugar de aprender una política por refuerzo, se **evoluciona una red de neuronas con NEAT** (*NeuroEvolution of Augmenting Topologies*), que optimiza a la vez los pesos y la topología de la red.

Además, el guía incorpora el modo de conducción que la Práctica 1 dejaba fuera: **las paradas aleatorias**. El seguidor tiene que detenerse detrás de él sin chocar y reanudar la marcha cuando el guía vuelva a arrancar.

El trabajo tiene tres partes: evolucionar los controladores en simulación, evaluarlos con el protocolo común del anexo B y **compararlos con las políticas de aprendizaje por refuerzo de la Práctica 1**. Como entonces, el mejor controlador se ejecuta después en el entorno real, sin volver a evolucionarlo.

2. Qué se mantiene y qué cambia respecto a la Práctica 1

Las prácticas son acumulativas: la Práctica 2 parte del software de la Práctica 1 (el entorno de Gymnasium, la comunicación con ROS 2, la caracterización de los sensores, el archivo de calibración y el evaluador del anexo B), que se reutiliza y se adapta.

	Práctica 1	Práctica 2
Controlador	Política de Stable-Baselines3: red de arquitectura fija	Red evolucionada con NEAT-Python: pesos y topología
Señal que guía la búsqueda	Recompensa en cada paso	Calidad: un valor por individuo evaluado
Uso del entorno	SB3 llama a reset() y step()	El mismo entorno; el bucle de evaluación de cada individuo lo escribe cada grupo

	Práctica 1	**Práctica 2**
Coste	Pasos del entorno	Individuos × generaciones × episodios por individuo, en tiempo real
Guía	Sin paradas (C1–C3)	C1–C3 y, además, con paradas (C4)
Observación y acción	Sensores de a bordo; acción continua	Sin cambios: mismas restricciones
Evaluación final	Anexo B, tres configuraciones	Anexo B, cuatro configuraciones y mismas métricas

3. El escenario y las configuraciones del guía

Sigue vigente todo lo descrito en el apartado 2 de la Práctica 1: el guía circula por un carril del anillo exterior girando siempre a la izquierda, lleva la pelota verde en su parte trasera, no reacciona al seguidor y se configura con un modo y unos límites de velocidad entre 10 y 40, en las unidades del simulador.

- **Modo variable con paradas**, nuevo en esta práctica: el guía fluctúa entre las velocidades mínima y máxima, frena en las curvas y, además, se detiene en puntos aleatorios del recorrido durante un tiempo, según la probabilidad y los tiempos de parada que se configuran en el simulador.
- Una parada incluye una dificultad que en la Práctica 1 no se exigía: el seguidor no puede limitarse a acompañar el ritmo del guía, tiene que **frenar hasta detenerse** a una separación segura y **volver a arrancar** cuando el guía lo haga, sin perderlo de vista.
- Las cuatro configuraciones de la evaluación están en el anexo B. Las tres primeras son las de la Práctica 1, lo que permite comparar directamente los resultados de ambas técnicas.

4. Herramientas

- **Gymnasium**: el entorno de la Práctica 1, reutilizado y, si hace falta, adaptado.
- **NEAT-Python**: implementación de NEAT, ya instalada en el contenedor. Documentación en https://neat-python.readthedocs.io. Se recomienda empezar por el ejemplo XOR.
- **robobo_ros2** y su módulo **sim**: igual que en la Práctica 1. El nodo puente sigue siendo la única vía de acceso a los sensores y actuadores del seguidor (interfaz en el anexo A de la Práctica 1).
- **Matplotlib** u otra biblioteca de gráficos, para las figuras de la memoria.

Stable-Baselines3 no interviene en la obtención del controlador de esta práctica: sólo se usa para volver a ejecutar las políticas de la Práctica 1 en la comparación del apartado 6.5.

No se proporciona código de partida. Junto a este enunciado se publica un documento de **indicaciones técnicas** que explica cómo integrar NEAT-Python con el entorno de la Práctica 1, comenta el archivo de configuración y ayuda a estimar el tiempo de cómputo. El uso de asistentes de IA para construir el software está permitido, pero debe explicarse en un documento específico, y se realizará una defensa para valorar el grado de conocimiento del código entregado. El estudiante es el autor de la práctica, no el asistente de IA, por lo que debe ser capaz de modificarlo y arreglarlo sin ella.

5. Observación, acción y función de calidad

> **La observación sigue limitada a los sensores del seguidor, recibidos a través de ROS 2, y la acción sigue siendo continua.** La red evolucionada recibe la observación y devuelve la acción en cada paso, exactamente como la política de la Práctica 1, y debe poder ejecutarse en el robot real.

Se puede partir de los espacios de observación y de acción de la Práctica 1 o rediseñarlos; los cambios se justifican en la memoria.

La función de calidad

La función de calidad puntúa el comportamiento de un individuo a lo largo de uno o varios episodios completos. En este caso, **puede calcularse con información privilegiada del simulador**: la separación real, la posición respecto al carril o el desenlace del episodio.

En la P1 se recomendaba que la recompensa dependiese de magnitudes inferibles porque el crítico aprende a predecirla a partir de la observación. En la evolución no hay crítico: la calidad sólo sirve para comparar individuos entre sí y decidir cuáles se reproducen, y la red nunca la recibe. Hay dos matices que conviene tener presentes:

- **La evolución optimiza la calidad, no la tarea.** Cualquier atajo que puntúe bien será encontrado y explotado. La calidad tiene que medir lo que se quiere obtener, y hay que comprobar en los vídeos que el comportamiento evolucionado es el esperado.
- **Una calidad con datos privilegiados sólo existe en simulación.** Impide evolucionar directamente sobre el robot real, que es lo que plantea la evolución *embodied* vista en teoría. Se puede comentar en la memoria qué parte de la calidad podría calcularse a bordo.

Para cada término de la calidad hay que dar en la memoria su fórmula, el valor numérico de cada coeficiente, la magnitud de la que depende y por qué está ahí. Como se vio en teoría, los pesos relativos de los términos pueden conducir a óptimos locales distintos; se valorará comparar al menos dos formulaciones y los comportamientos a los que dan lugar.

6. Desarrollo

6.1. Integración de NEAT con el entorno

Cada individuo de la población es un genoma que se traduce en una red. La red controla al seguidor durante uno o varios episodios del entorno de Gymnasium, y de ese comportamiento sale su calidad. Hay que definir y justificar en la memoria:

- **Red**: número de entradas y salidas, funciones de activación y conectividad inicial. El rango de las salidas debe ser coherente con el espacio de acción.
- **Parámetros del algoritmo evolutivo**: tamaño de la población, número de generaciones, umbral de especiación, estancamiento, elitismo y probabilidades de mutación de pesos y de estructura. Se puede partir de los valores de las indicaciones técnicas; los que se cambien se justifican.
- **Evaluación de cada individuo**: duración del episodio de evaluación, número de episodios por individuo, configuración del guía durante la evolución y cómo queda el escenario al empezar cada episodio.
- **Función de calidad**, según el apartado 5.

6.2. Presupuesto de cómputo

Cada evaluación transcurre en tiempo real, y NEAT necesita muchas: el coste de una evolución es, como máximo, el número de generaciones por el tamaño de la población, por los episodios de cada individuo y por la duración de cada episodio. **El tiempo es el recurso que más limita esta práctica**, y gestionarlo forma parte del trabajo:

- Antes de lanzar una evolución larga, se mide cuánto dura una generación y se estima el total. Como referencia, con pasos de 0,2 s, una población de 20 individuos y episodios de 30 s como máximo, una generación dura unos 11 minutos en el peor caso, y 30 generaciones, hasta unas 5 horas y media; en la práctica algo menos, porque muchos episodios terminan antes.
- Se recomienda planificar evoluciones que no superen las **4 a 6 horas**, que puedan quedar en marcha sin supervisión y que se reanuden desde un punto de control si se interrumpen.
- Los dos ordenadores de la pareja pueden evolucionar en paralelo, con semillas o configuraciones distintas. Cada simulador admite una sola evaluación a la vez.
- No es obligatorio evolucionar un controlador distinto para cada configuración del guía: **se valora obtener un único controlador que resuelva varias o todas**, lo que además ahorra evoluciones.

La memoria declara, para cada evolución, el número de generaciones, el de evaluaciones, el de pasos del entorno y las horas de reloj que ha costado.

6.3. Evolución

Las cuatro configuraciones del guía del anexo B tienen dificultad creciente, y **la calificación irá en proporción a cuáles resuelva cada grupo**. Para cada configuración resuelta (o para el controlador único que resuelva varias) hay que:

- Evolucionar la configuración final con **al menos dos semillas** distintas, y preferiblemente tres.
- Seleccionar el controlador final. El mejor individuo de una evolución puede deber su calidad a un episodio afortunado: conviene volver a evaluar a los mejores candidatos antes de elegir.
- Tener una **línea base**. La población inicial, formada por redes aleatorias, ya hace de referencia. Se recomienda además evaluar con la misma calidad el controlador programado a mano en el reto de las primeras semanas.

6.4. Evaluación con el protocolo común

El controlador final de cada configuración se evalúa con el **protocolo del anexo B**, igual al de la Práctica 1 con una configuración más. El evaluador ya existe: es el de la Práctica 1, ampliado con C4. Las métricas deben calcularse exactamente como se definen, para que los resultados sean comparables entre grupos y con la Práctica 1.

6.5. Comparación con la Práctica 1

Es la parte central del análisis. En la misma sesión de trabajo, con la misma versión del simulador y el mismo multiplicador de velocidad, se evalúan con el anexo B la política final de la Práctica 1 y el controlador final de la Práctica 2 en C1, C2 y C3, y se comparan en:

- **Rendimiento**: las métricas del anexo B por configuración.
- **Coste**: pasos del entorno y horas de reloj consumidos por el entrenamiento y por la evolución.
- **Tamaño**: número de parámetros de la política de Stable-Baselines3 frente al número de nodos y conexiones de la red evolucionada.

- **Interpretabilidad**: las redes de NEAT suelen ser pequeñas. Se pide **interpretar la red final**: qué hace con cada entrada, qué conexiones llevan la información de la pelota a cada actuador y en qué se parece o se diferencia del controlador programado a mano en el reto.

Si alguna configuración no se resolvió en la Práctica 1, se compara con lo que se obtuvo entonces.

6.6. Transferencia al entorno real

El mejor controlador se ejecuta en la ciudad del laboratorio 0.0 de la FIC **sin volver a evolucionarlo**, con el archivo de calibración del robot real de la Práctica 1. Basta con las configuraciones que se puedan reproducir en el laboratorio. Hay que documentar qué se degrada, por qué, y si la red evolucionada transfiere mejor o peor que la política de la Práctica 1. Como entonces, **se evalúa el análisis, no el éxito**.

7. Consideraciones y posibles errores

Los siguientes fenómenos pueden aparecer durante el desarrollo. Cómo tratarlos forma parte del trabajo, y la memoria debe mostrar cómo se han identificado y abordado.

- **La calidad es ruidosa.** El mismo genoma puede obtener calidades distintas en dos evaluaciones: el guía variable, las paradas aleatorias y los retardos de la comunicación cambian cada episodio. NEAT-Python vuelve a evaluar a todos los individuos en cada generación, incluidos los que pasan intactos por elitismo, así que la mejor calidad de una generación puede bajar respecto a la anterior.
- **El coste crece a medida que la población mejora.** Los individuos malos fallan en pocos segundos y los buenos agotan el episodio, de modo que las últimas generaciones son las más lentas. El presupuesto se estima con la duración máxima del episodio, no con la de la primera generación.
- **La evolución explota la calidad.** Si un comportamiento no deseado puntúa bien, aparecerá: girar sobre sí mismo para no perder la pelota, quedarse justo en el límite de la separación permitida o detenerse si eso evita el fallo. Si aparece alguno, se describe en la memoria junto con la corrección.
- **La población inicial no sabe moverse.** Con redes mínimas y pesos aleatorios, muchos individuos fallan casi de inmediato. Si la calidad les da a todos el mismo valor, la selección es aleatoria: debe distinguir entre individuos malos y menos malos desde la primera generación.
- **La especiación depende del tamaño de la población.** Con poblaciones pequeñas, un umbral de compatibilidad alto deja una sola especie y la innovación deja de estar protegida; uno bajo produce especies de uno o dos individuos. El número de especies a lo largo de la evolución se vigila y se comenta.
- **Las evoluciones largas se interrumpen.** Una pérdida de conexión con el nodo puente deja la evolución detenida sin mensaje de error, como ya ocurría en la Práctica 1. Hay que guardar un punto de control en cada generación y poder reanudar desde él.
- **Siguen vigentes las consideraciones de la Práctica 1**: el guía no espera, las curvas, las frecuencias y retardos medidos en tiempo real, un único blob por color y el efecto del multiplicador de velocidad del simulador.

8. Condiciones

- El controlador final es una red obtenida con NEAT-Python. **No se admiten retoques a mano** de sus pesos ni de su estructura. El preprocesado de la observación y el escalado de la acción dentro del entorno sí están permitidos, y se declaran.
- La observación contiene sólo información de los sensores del seguidor recibida por ROS 2, y la acción es continua.

- La calidad puede usar información privilegiada del simulador, declarando cuál.
- El nodo puente se lanza sólo con los módulos necesarios.
- El multiplicador de velocidad del simulador se declara en la memoria y es el mismo en la evolución y en la evaluación. Si difiere del usado en la Práctica 1, se indica en la comparación.
- Las semillas aleatorias de cada evolución se fijan y se declaran.
- Los parámetros que dependen de la calibración siguen en un archivo de configuración.
- El protocolo de evaluación del anexo B se sigue sin modificaciones.

9. Elementos a entregar

Un archivo .zip con cuatro cosas:

9.1. Memoria

En formato PDF, de seis páginas como máximo más los anexos que hagan falta, en Calibri de 11 pt con interlineado sencillo: texto justificado, figuras y tablas numeradas con pie, valores numéricos explícitos en la función de calidad y en los parámetros de NEAT, y todos los resultados comentados. La estructura debe ser:

1. **Diseño**: espacios de observación y de acción (y sus cambios respecto a la Práctica 1), tipo de red, función de calidad, evaluación de cada individuo y tabla con los parámetros de NEAT utilizados.
2. **Evolución** para cada configuración resuelta: curvas de evolución, especiación y coste (generaciones, evaluaciones, pasos del entorno y horas).
3. **La red obtenida**: figura de la topología de cada controlador final y su interpretación.
4. **Resultados**: tablas de métricas del anexo B para la línea base y para el controlador final de cada configuración, y la comparación con la Práctica 1 del apartado 6.5.
5. **Transferencia al entorno real.**
6. **Conclusiones**: aprendizaje por refuerzo frente a neuroevolución en este problema, problemas encontrados y mejoras.

Curvas de evolución

Una curva de evolución muestra cómo cambia la población a lo largo de las generaciones. Sirve para comprobar que la evolución progresa, detectar el estancamiento y comparar configuraciones. Cada evolución de la solución final se acompaña de curvas que cumplan los requisitos siguientes.

- **Magnitudes**: como mínimo, la calidad del mejor individuo y la calidad media de la población en cada generación, con la desviación típica de la población como banda. Se recomienda añadir el número y el tamaño de las especies, en un gráfico de áreas apiladas, y la complejidad del mejor individuo (nodos y conexiones activas).
- **Eje horizontal**: generaciones. El pie de la figura indica el tamaño de la población, los episodios por individuo, la duración máxima del episodio y el total de pasos del entorno y de horas.
- **Eje vertical**: nombre de la magnitud y su escala. La calidad es adimensional, pero su rango teórico se indica en el texto.
- **Semillas**: la configuración final se evoluciona con al menos dos semillas. La figura muestra la media y una banda de dispersión, indicando si corresponde al mínimo y el máximo o a la desviación típica.
- **Referencia**: se recomienda una línea horizontal con la calidad que obtiene el controlador programado a mano en el reto.

- **Origen de los datos**: los valores se registran durante la evolución, por ejemplo con un archivo CSV con una fila por generación. Las figuras se generan con código propio a partir de esos datos. Se puede partir del archivo `visualize.py` de los ejemplos de NEAT-Python; las capturas de la consola no se admiten como figuras.
- **Presentación**: cada figura va numerada y con pie, y se comenta en el texto: cuándo la población supera a la referencia, si se estanca y qué ocurre con las especies.

> Una curva de evolución no sustituye a la evaluación. La calidad es la de cada grupo y se mide con pocos episodios por individuo; el protocolo del anexo B es común a todos los grupos y es el que mide la tarea. Una calidad alta con episodios sin completar delata una calidad mal planteada.

9.2. Evidencias

Las figuras y las tablas de la memoria deben poder rehacerse a partir de lo que se entrega: los archivos de configuración de NEAT, los genomas finales con el programa que los carga y los ejecuta, el último punto de control de cada evolución final, el registro por generación de cada evolución y el registro por paso de cada evaluación, junto con el código que los lee. No hay un formato obligatorio.

Se añaden, además, vídeos de la ejecución de cada controlador final en las configuraciones resueltas y de la transferencia al entorno real.

9.3. Declaración del uso de IA

Documento aparte, de una página como máximo, indicando qué herramientas de IA se han utilizado y para qué. Se debe dejar por escrito cuál ha sido la contribución propia y cuál la de la IA.

9.4. Código fuente comentado

Se incluye el código fuente utilizado, con instrucciones para reproducir la evolución y para ejecutar el controlador final de cada configuración.

Anexo A. Interfaz de ROS 2

Es la misma del anexo A de la Práctica 1. Para el nuevo modo del guía no hace falta ningún recurso adicional: las paradas se configuran en el simulador, y la posición del guía, necesaria para la calidad y para las métricas, se obtiene como entonces con un segundo contenedor que sólo carga el módulo sim.

Anexo B. Protocolo de evaluación

Es el protocolo de la Práctica 1 con una configuración más (C4) y una métrica adicional para ella. Se reproduce completo para que el documento sea autosuficiente.

B.1. Magnitudes de referencia

Magnitud	Definición
Pelota verde	Referencia de todas las medidas, porque es lo que el seguidor percibe. Su posición sale de la pose del robot guía desplazada hacia su parte trasera
Separación	Distancia entre el centro del seguidor y la pelota verde
Banda objetivo	Separación entre 0,20 m y 0,45 m (en caso de usar otra, se debe justificar en la memoria)
Desviación	Ángulo entre el rumbo del seguidor y la dirección a la pelota. Corresponde a la x del blob, y comparar las dos comprueba la calibración
Parada del guía	Intervalo en el que la velocidad del guía, calculada a partir de su pose, es inferior a 0,01 m/s durante más de 0,5 s
Parámetros	Pelota 0,08 m por detrás del centro del guía · colisión por debajo de 0,12 m · guía perdido por encima de 0,90 m (en caso de usar otros, se deben justificar en la memoria)

NOTA: en la versión 4.3.0 de RoboboSim es posible detectar la posición del robot guía, por lo que se podría usar esta posición y no la de la pelota verde para las medidas.

B.2. Condiciones de la evaluación

Cada episodio empieza con un reinicio de la simulación: ambos robots parten de donde los coloca el escenario y el seguidor parte parado, aunque se admiten ciertas acciones “de arranque” predeterminadas. Lo único que se fija es la configuración del guía. Se analizan ejecuciones de 120 segundos con estos valores para el guía; C4 se ejecuta dos veces por el carácter aleatorio de las paradas:

Configuración	Modo	Velocidad mínima	Velocidad máxima	Ejecuciones
C1. Lenta constante	Crucero	10	10	1
C2. Contraste	Crucero	10	40	1
C3. Fluctuante	Variable	10	40	1
C4. Con paradas	Variable con paradas	10	40	2

En C4, la probabilidad de parada y los tiempos de parada son [valores por fijar], iguales para todos los grupos.

B.3. Fin de cada episodio

Cada episodio dura 120 s y se da por completado si en ese tiempo no se produce ninguno de estos fallos, que se comprueban en cada paso:

- **Colisión**: separación menor que 0,12 m (en caso de usar otra, se debe justificar en la memoria).
- **Guía perdido**: separación mayor que 0,90 m, o la pelota sin detectarse durante más de tres segundos seguidos (en caso de usar otra, se debe justificar en la memoria).

Una terminación del entorno propio por otra causa se registra como tal y no cuenta como completado. Salirse del carril no es fallo del protocolo, porque la geometría del carril no está disponible: se juzga en los vídeos.

B.4. Registro y métricas

En cada paso se registra el instante, la posición del seguidor y de la pelota (o del guía), la separación, la desviación lateral y la acción aplicada. De ahí salen las métricas, en conjunto y desglosadas por configuración:

Métrica	Definición
Completados	Número y porcentaje de episodios completados
Desenlaces	Número de episodios terminados por cada una de las causas de B.3
Separación	Porcentaje de pasos dentro de la banda objetivo, y separación media
Desviación	Media del valor absoluto, en grados, y porcentaje de pasos por debajo de 15°
Paradas (sólo C4)	Número de paradas del guía y número de ellas superadas: el seguidor se detiene sin colisión y sigue sin fallo 10 s después de que el guía reanude la marcha